

```

%%capture
!apt-get install -y -qq software-properties-common python-software-properties module-init-
!add-apt-repository -y ppa:fenics-packages/fenics
!apt-get update -qq
!apt install -y --no-install-recommends fenics
!rm -rf *
from fenics import *

```

▼ 2D Problem

1. Only Dirichlet BC
2. $\mathbb{P}_1$ and $\mathbb{P}_2$ elements
3. *preference* must be fixed

```

def stokes(n, f, u_bc, p_exact):
    mesh = UnitSquareMesh(n, n, 'crossed')

    V = VectorElement('CG', mesh.ufl_cell(), degree=2)
    Q = FiniteElement('CG', mesh.ufl_cell(), degree=1)

    X = FunctionSpace(mesh, V*Q)

    u, p = TrialFunctions(X)
    v, q = TestFunctions(X)

    a = (inner(grad(u), grad(v)) - div(v)*p + div(u)*q)*dx
    L = dot(f, v)*dx

    def boundary(x, on_boundary):
        return on_boundary

    def origin(x, on_boundary):
        return near(x[0], 0) and near(x[1], 0)

    bc = DirichletBC(X.sub(0), u_bc, boundary)
    bc_pressure = DirichletBC(X.sub(1), p_exact(0, 0), origin, 'pointwise')
    x = Function(X)
    solve(a==L, x, [bc, bc_pressure])

    u, p = x.split()

    return mesh, u, p

```

Example

```

n=2
u_exact = Expression((
    '-cos(x[0]) * sin(x[1])',
    'sin(x[0]) * cos(x[1])'
))

```

```

    ), degree=3)
p_exact = Expression(
    '-0.25 * (cos(2*x[0]) + cos(2*x[1]))',
    degree=3)
f = Expression((
    '-2 * cos(x[0]) * sin(x[1]) + 0.5 * sin(2 * x[0])',
    '2 * sin(x[0]) * cos(x[1]) + 0.5 * sin(2 * x[1])'
    ), degree=3)

for degree in [1, 2]:
    for n in [5, 10, 20, 40]:
        mesh, uh, ph = stokes(n, f, u_exact, p_exact)

        eL2 = errornorm(p_exact, ph, 'L2')
        eH1 = errornorm(u_exact, uh, 'H1')

        print('n={} degree={} eL2={:.2e} eH1={:.2e}'.format(n, degree, eL2, eH1))
    print()

☞ n=5 degree=1 eL2=3.98e-03 eH1=1.60e-03
   n=10 degree=1 eL2=1.01e-03 eH1=3.99e-04
   n=20 degree=1 eL2=2.55e-04 eH1=9.97e-05
   n=40 degree=1 eL2=6.38e-05 eH1=2.49e-05

   n=5 degree=2 eL2=3.98e-03 eH1=1.60e-03
   n=10 degree=2 eL2=1.01e-03 eH1=3.99e-04
   n=20 degree=2 eL2=2.55e-04 eH1=9.97e-05
   n=40 degree=2 eL2=6.38e-05 eH1=2.49e-05

```

▼ 3D Problem

1. Only Dirichlet BC
2. $\mathbb{P}_1$ and $\mathbb{P}_2$ elements
3. A preconditioner is defined

```

def ThreeDimStokes(n, f, u_r, u_t, u_b):
    mesh = UnitCubeMesh(n, n, n)

    V = VectorElement('Lagrange', mesh.ufl_cell(), degree=2)
    Q = FiniteElement('Lagrange', mesh.ufl_cell(), degree=1)
    X = FunctionSpace(mesh, V*Q)

    u, p = TrialFunctions(X)
    v, q = TestFunctions(X)

    a = (inner(grad(u), grad(v)) - div(v)*p + div(u)*q)*dx
    L = (inner(f, v))*dx

    def right(x, on_boundary):
        return x[0]>1.0-DOLFIN_EPS
    def top(x, on_boundary):

```

```

    return x[1]>1.0-DOLFIN_EPS
def bottom (x, on_boundary):
    return x[0]<DOLFIN_EPS

bc_r = DirichletBC(X.sub(0), u_r, right)
bc_t = DirichletBC(X.sub(0), u_t, top)
bc_b = DirichletBC(X.sub(0), u_b, bottom)

A, bb = assemble_system(a, L, [bc_r, bc_t, bc_b])

# Predconditioner
b = inner(grad(u), grad(v))*dx + p*q*dx
P, bp = assemble_system(b, L, [bc_r, bc_t, bc_b])

x = Function(X)

#Krylov solver and AMG preconditioner
solver = KrylovSolver("default", "amg")
solver.set_operators (A, P)

solver.solve(x.vector(), bb)
u, p = x.split()
return u, p

```

The condition imposed are:

1. No slip condition in the upper and lower part
2. Inflow on the right $u = \{-\sin(\pi y), 0, 0\}$

```

inflow = Expression((" -sin(x[1]*pi)", "0.0", "0.0"), degree=2)
u_noSlip = Constant((0, 0, 0))
f = Constant((0, 0, 0))
n = 16

u, p = ThreeDimStokes(n, f, inflow, u_noSlip, u_noSlip)
ufile_pvd = File("velocity.pvd")
ufile_pvd << u
pfile_pvd = File("pressure.pvd")
pfile_pvd << p

```

